



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

课程报告

开课学期: 2026 夏季

课程名称: 计算机设计与实践

项目名称: 支持 miniRV/LA 的 SoC 设计

项目类型: 综合设计型

课程学时: 48 地点: T2210

学生班级: 12、12

学生学号: 2024311488、2024311234

学生姓名: 张正翰、何文杰

评阅教师: _____

报告成绩: _____

实验与创新实践教育中心制

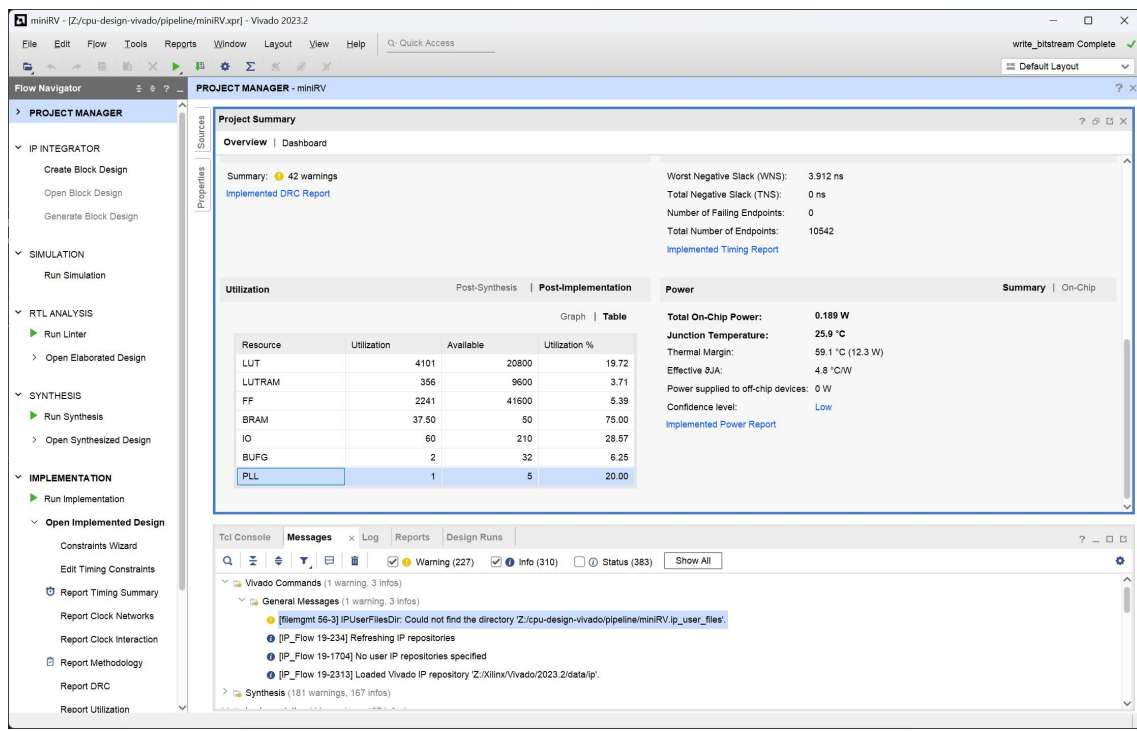
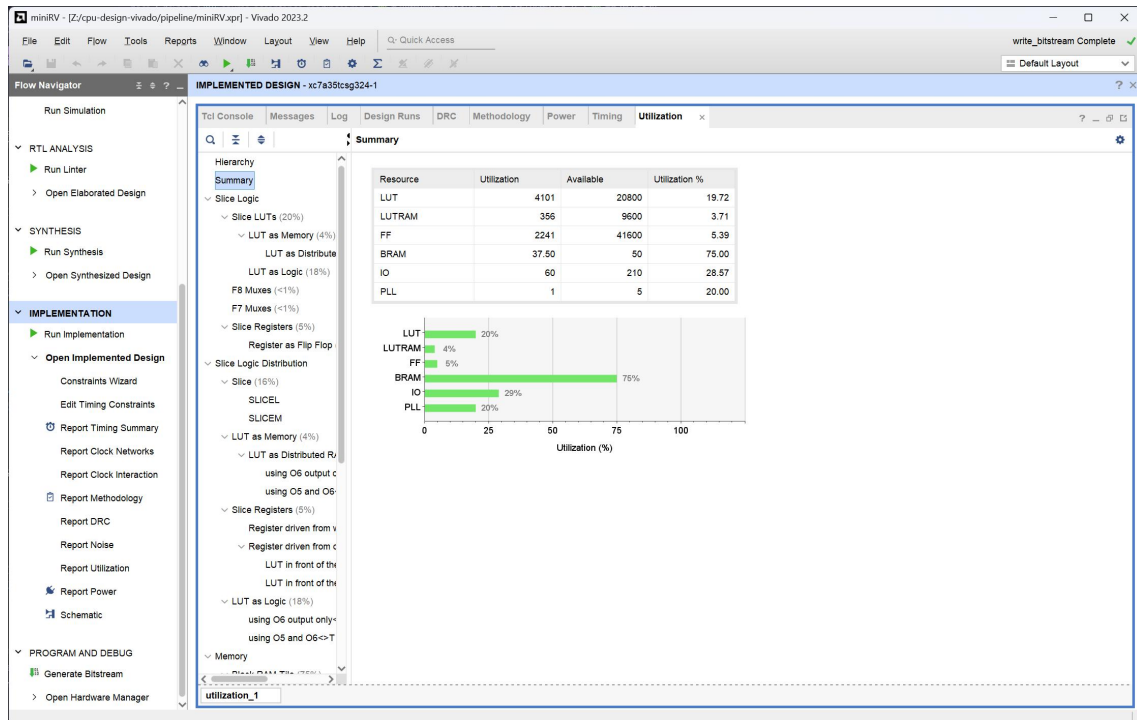
2026 年 7 月

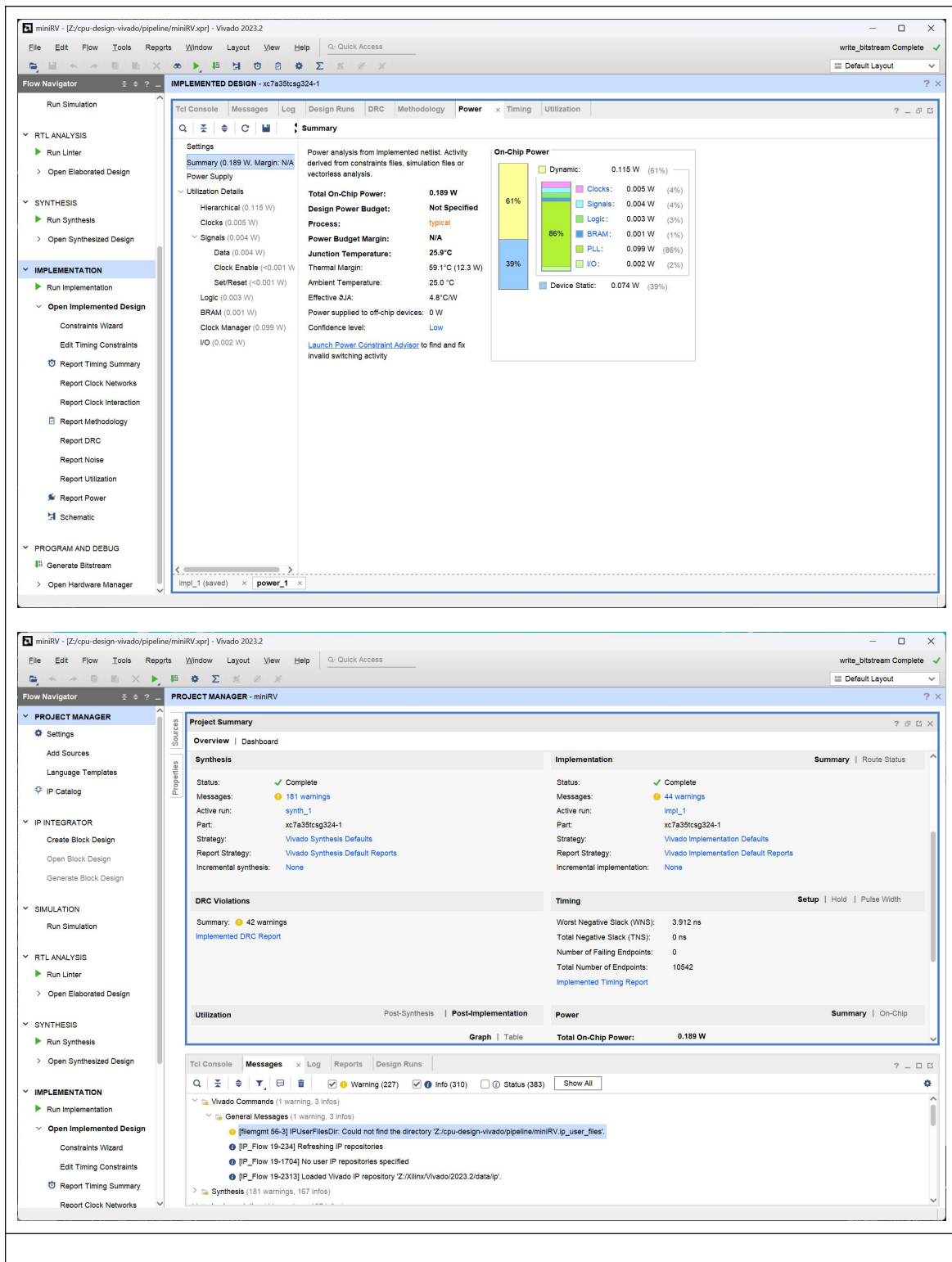
设计概述			
组号	姓名	完成的指令	完成的其他内容
	张正翰	包括移位运算、加减算术运算、部分逻辑运算、Load-Store、直接跳转在内的 A 组指令	1. 单周期 CPU (Lab 1-General): 完整数据通路 + 控制器, 独立设计并验证的 Booth 乘法器与恢复余数除法器。 2. SoC (Lab 2-B): 直接映射 ICache 与写直通/非写分配 DCache (各 1 KiB、16 B 行)、AXI4 主机桥、AXI 互连(主存 + MMIO 分流)、五类外设 (拨码开关、LED、数码管、UART、64 位计时器)。 3. 板上程序: C_TEST 0~2 (UART 回显、格式化 I/O、排序) 与 CoreMark 移植, freestanding RV32IM/ILP32 运行时。
	何文杰	包括乘除逻辑运算、比较逻辑运算、部分逻辑运算、条件分支在内的 B 组指令	1. 流水线 - CPU (Lab 2-B) : IF/ID/EX/MEM/WB 五级流水, 两级前递网络 + WB→ID 读端口旁路、load-use 暂停、EX 级分支解析与冲刷。 2. 验证体系: 45 项官方 Trace × 10 个稳定配置全部通过; unit / integration / system 三层仓库测试; 单周期 SoC 已完成 50 MHz clean 实现与 EGO1 板测。 3. 工程化: 配置矩阵驱动的双产品构建与验证 CLI(Justfile), 设计合同 CSV 先于 RTL 的门禁流程。
设计的主要特色 (除基本要求以外的设计)			
1. 配置矩阵驱动的双产品工程。单周期与流水线各自持有唯一 HDL 真相源, 拓扑 (历史 ROM/RAM、AXI 直连、完整 SoC) 与 Cache 使能全部通过编译宏选择, 不 fork 文件、不改 defines.vh。config/build-configs.tsv 固定 10 个稳定配置, 每个配置的产品、拓扑、存储模型、Cache、后端与产物目录都被校验, 非法组合直接拒绝。 2. 设计合同先于 RTL 的门禁。单周期的数据通路表与控制信号表、流水线的段寄存器表/冒险表/流控表都是 CSV 形式的强制前置产物, 控制符号必须与 defines.vh 宏名一致, 有副作用的控制信号不允许留空。RTL 与 CSV 逐字段对应, 便于交叉复核。 3. 分层验证体系与明确的证明边界。unit (单模块)、integration (单接口切片)、trace (提交与访存观测)、system (CPU 驱动的整机仿真) 四层各自写明"不证明什么"; closure 门禁一次性跑完格式检查、10 配置 lint、10×45 Trace、全部 unit/integration/system 与工程文件 XML 校验。 4. 总线协议的严格化。MMIO 只接受单拍 32 位 INCR 事务, 普通 Cache 缺失使用四拍整行回填; 被拒绝事务在形状检查通过前不产生任何本地读写使能, 因此非法 UART 读不会弹出 FIFO、非法 LED 写不会改变输出, 并按 AXI 规定返回形状一致的 DECERR。 5. 真实的子字访存契约。lb/lbu/lh/lhu/sb/sh 一路贯通到 MMIO: 读保留原始字节地址并由 CPU 按低位选择, 写只接受 MREQ 实际可产生的地址与 strobe 组合, 其余组合稳定报错且无副作用; 测试对四种地址低位 × 16 种 strobe 做了 64 组合穷举。 6. 有测量支撑的流水线性能优化。以单次迭代 CoreMark 为基准建立周期、IPC 与停顿归			

因计数器，两轮优化 (Booth 加法与移位融合为一拍；取指每拍一次请求，含 ifetch_ready 握手与 IF/ID 后一级 skid buffer) 累计降低 34.3% 周期数，IPC 由 0.192 提升到 0.293。

7. 可审计的板级交付。bitstream、程序镜像、COE、manifest 逐项记录 SHA-256 与源提交，确保 XCI 请求的 COE 与实际消费的 COE 一致。

流水线 SoC 在实现后 (Post Implementation) 的资源使用、功耗数据截图 (要求**不能有时序违例**)





	无符号除法器), 以 <code>busy</code> 向外表达"结果未就绪"
MREQ	访存请求生成。按地址低两位生成字节使能, 并把写数据左移到目标字节位置; 非对齐的半字/字访问直接抑制请求
MEXT	载入数据对齐与扩展。先按字节偏移把目标字节/半字右移到低位, 再按 <code>ram_rop</code> 做 <code>lb/lbu/lh/lhu/lw</code> 的符号或零扩展
Inst_ROM	指令存储器 (历史 Basic 路径)。BRAM IP 封装, <code>inst_valid</code> 为 <code>inst_req</code> 延迟一拍
Data_RAM	数据存储器 (历史 Basic 路径)。BRAM IP 封装, <code>data_valid</code> / <code>data_wresp</code> 分别为读/写使能延迟一拍

三、模块间连线的信号名与位宽

下表即为数据通路图中每条模块间连线的依据, 与 `design/single_cycle/datapath.csv` 的累积 `complete` 行一一对应。

连线 (目的端口)	位宽	来源
PC.npc	32	NPC.npc
NPC.pc	32	PC.pc
NPC.offset	32	`SEXT.ext` 或 `ALU.c` (JALR 用 ALU.c)
NPC.br	1	ALU.br
RF.rR1	5	<code>inst[19:15]</code>
RF.rR2	5	<code>inst[24:20]</code>
RF.wR	5	`inst[11:7]` (多周期完成时用锁存的目标寄存器号)
RF.wD	32	`ALU.c` / `SEXT.ext` / `MEXT.ext` / `NPC.pc4` (由 <code>rf_wsel</code> 四选一)
SEXT.imm	25	<code>inst[31:7]</code>
ALU.a	32	`RF.rD1` 或 `PC.pc` (<code>alua_sel</code> , AUIPC 用 <code>pc</code>)
ALU.b	32	`RF.rD2` 或 `SEXT.ext` (<code>alub_sel</code>)
MREQ.ram_addr	32	ALU.c
MREQ.ram_wdata	32	RF.rD2
MEXT.din	32	<code>daccess_rdata</code>
MEXT.byte_offs	2	`ALU.c[1:0]` (锁存)
Controller.opcode/funct3/funct7	7/3/7	<code>inst[6:0]</code> / <code>inst[14:12]</code> / <code>inst[31:25]</code>

四、控制信号

控制信号全部由 Controller 组合产生, 编码宏定义集中在 `defines.vh`, 图中标注的符号名与宏名一致 (去掉反引号)。

控制信号	位宽	含义与取值
<code>npc_op</code>	2	NPC_PC4 / NPC_JALR / NPC_BRA / NPC_JMP
<code>sext_op</code>	3	EXT_I / EXT_S / EXT_B / EXT_U / EXT_J
<code>alua_sel</code>	1	ALU_A_RS1 / ALU_A_PC
<code>alub_sel</code>	1	ALU_B_RS2 / ALU_B_EXT
<code>alu_op</code>	5	ALU_ADD...ALU_REMU 共 21 种编码 (含比较与乘除)

is_mul` `is_div	/	1/1	启动长延迟乘法 / 除法单元
ram_rop		3	RAM_EXT_N / W / B / BU / H / HU
ram_wop		4	RAM_WE_N(0000) / B(0001) / H(0011) / W(1111)
rf_we		1	寄存器写使能 (译码级意图, 实际提交由 commit_rf_we 给出)
rf_wsel		2	WB_ALU / WB_RAM / WB_PC4 / WB_EXT

1.2 单周期 CPU 详细设计

一、各部件接口信号表

1) PC —— 程序计数器

信号	方向	位宽	功能描述
clk	input	1	时钟
rst	input	1	高有效复位, 复位后 pc = PC_INIT_VAL = 0x00000000
npc	input	32	下一条指令地址
fetch	input	1	更新使能, 仅当本条指令完成 (inst_done) 时为 1
pc	output	32	当前指令地址

2) NPC —— 下一 PC 生成

信号	方向	位宽	功能描述
op	input	2	NPC_PC4 / NPC_JALR / NPC_BRA / NPC_JMP
pc	input	32	当前 PC
offset	input	32	分支/跳转偏移; JALR 时为 ALU 算出的目标地址
br	input	1	ALU 给出的分支判定
npc	output	32	下一条指令地址
pc4	output	32	pc + 4, 供 jal/jalr 写回链接地址

3) RF —— 通用寄存器堆

信号	方向	位宽	功能描述
clk	input	1	时钟
rR1` / `rR2	input	5 / 5	两个读地址, 来自 inst[19:15] 与 inst[24:20]
we	input	1	写使能, 实际接提交级 commit_rf_we
wR	input	5	写地址
wD	input	32	写数据
rD1` / `rD2	output	32 / 32	组合读出, rR = 0 时恒为 0

4) SEXT —— 立即数扩展

信号	方向	位宽	功能描述
op	input	3	EXT_I / EXT_S / EXT_B / EXT_U / EXT_J
imm	input	25	inst[31:7]

ext	output	32	重排并符号扩展后的 32 位立即数
-----	--------	----	-------------------

5) Controller —— 主译码器

信号	方向	位宽	功能描述
opcode	input	7	inst[6:0]
funct3	input	3	inst[14:12]
funct7	input	7	inst[31:25]
npc_op	output	2	PC 更新方式
sxt_op	output	3	立即数格式
alua_sel` `alub_sel	/ output	1 / 1	ALU 两个操作数来源
alu_op	output	5	ALU 功能码
is_mul` `is_div	/ output	1 / 1	启动乘法 / 除法单元
ram_r_op	output	3	载入扩展方式, RAM_EXT_N 表示不访存
ram_w_op	output	4	存储字节使能模板, RAM_WE_N 表示不写
rf_we	output	1	寄存器写意图
rf_wsel	output	2	写回数据来源

6) ALU —— 算术逻辑与长延迟运算单元

信号	方向	位宽	功能描述
clk` / `rst	input	1 / 1	长延迟子单元需要时钟与复位
op	input	5	功能码, 含算术、逻辑、移位、比较、乘、除、求余
a` / `b	input	32 / 32	两个操作数
c	output	32	运算结果
br	output	1	分支判定 (EQ/NE/SLT/SLTU/GE/GEU)
busy	output	1	四个长延迟子单元 busy 的或, 表示结果尚未就绪

7) MREQ —— 访存请求生成

信号	方向	位宽	功能描述
ram_addr	input	32	访存地址, 来自 ALU.c
ram_rop	input	3	载入类型
ram_wop	input	4	存储类型
ram_wdata	input	32	待写数据, 来自 RF.rD2
da_ren	output	4	读字节使能
da_addr	output	32	访存地址 (直通)
da_wen	output	4	写字节使能
da_wdata	output	32	已左移对齐到目标字节位置的写数据

8) MEXT —— 载入数据对齐与扩展

信号	方向	位宽	功能描述
op	input	3	载入扩展方式 (锁存的 ram_rop)
din	input	32	存储器返回的 32 位端口值

byte_offs	input	2	锁存的地址低两位
ext	output	32	对齐并扩展后的写回数据

9) multiplier / divider —— 长延迟运算子单元 (参数化, 可独立复用)

信号	方向	位宽	功能描述
x` / `y	input	WIDTH	乘数/被乘数, 或除法的被除数/除数 (符号-数值格式)
start	input	1	电平启动
z	output	RESULT_WIDTH	乘积, 或商
r	output	WIDTH	余数 (仅除法器)
busy	output	1	运算进行中

二、控制信号编码与译码结构

全部编码集中在 defines.vh, Controller 引用宏名而不写裸常量, 这样设计表 CSV、RTL 与数据通路图上的符号完全一致。

编码组	取值
ALU_*	ADD/SUB/AND/OR/XOR/SLL/SRL/SRA = 5'h00~07, EQ/NE = 08/09, MUL/MULH/MULHU = 0A/0B/0C, SLT/SLTU/GE/GEU = 0D~10, DIV/DIVU/REM/REMU = 11~14
NPC_*	PC4 = 00, JALR = 01, BRA = 10, JMP = 11
EXT_*	I = 000, S = 001, B = 010, U = 011, J = 100
WB_*	ALU = 00, RAM = 01, PC4 = 10, EXT = 11
RAM_EXT_*	N = 000, W = 001, B = 010, BU = 011, H = 100, HU = 101
RAM_WE_*	N = 0000, B = 0001, H = 0011, W = 1111

Controller 采用"完整默认值 + 三层 case 覆写"的结构: 进入 always 块先把 11 个输出赋成无副作用默认值 (npc_op = NPC_PC4, rf_we = 0, ram_r_op = RAM_EXT_N, ram_w_op = RAM_WE_N, is_mul = is_div = 0), 再按 opcode 分组, 组内按 funct3、必要时再按 funct7 覆写。每一层都带 default 分支, 因此任何未识别编码都只会顺序执行到下一条指令、不产生任何架构副作用。指令按 opcode 分为 9 组:

opcode	类别	指令
0010011	I 型 ALU	addi slli slti sltiu xori srli srai ori andi
0110011	R 型 + M 扩展	add sub sll slt sltu xor srl sra or and / mul mulh mulhu div divu rem remu
0000011	载入	lb lh lw lbu lhu
0100011	存储	sb sh sw
1100011	条件分支	beq bne blt bge bltu bgeu
0110111	U 型	lui
0010111	U 型	auipc
1101111	J 型	jal
1100111	I 型跳转	jalr

三、关键实现之一：分支与跳转的 PC 生成 (NPC.v)

四路选择在一个 case 里完成。JALR 按 RISC-V 规定把目标地址最低位强制清零，条件分支在 br 为假时退化为顺序取指，因此分支不成立与普通指令走同一条路径。

```
assign pc4 = pc + 32'h4;
always @(*) begin
    case (op)
        `NPC_PC4: npc = pc4;
        `NPC_JALR: npc = {offset[31:1], 1'b0};
        `NPC_BRA: npc = br ? pc + offset : pc4;
        `NPC_JMP: npc = pc + offset;
        default: npc = pc4;
    endcase
end
```

四、关键实现之二：字节/半字存储的对齐 (MREQ.v)

字节使能与写数据左移量都由地址低两位导出。sb 把 4'h1 左移到目标字节；sh 只在偏移的最低位为 0 时才产生 4'h3 << offset；sw 只在偏移为 0 时产生 4'hF。非对齐的半字/字访问不会得到任何使能，即请求被直接抑制，不产生异常也不会破坏存储器内容。

```
wire [1:0] offset = ram_addr[1:0];
wire [4:0] data_shift = {offset, 3'b000};
always @(*) begin
    da_wen = 4'h0; // 默认不写
    da_wdata = ram_wdata;
    case (ram_wop)
        `RAM_WE_B: begin
            da_wen = 4'h1 << offset;
            da_wdata = ram_wdata << data_shift;
        end
        `RAM_WE_H: if (offset[0] == 1'b0) begin
            da_wen = 4'h3 << offset;
            da_wdata = ram_wdata << data_shift;
        end
        `RAM_WE_W: if (offset == 2'h0) da_wen = 4'hF;
        default: begin da_wen = 4'h0; da_wdata = ram_wdata; end
    endcase
end
```

五、关键实现之三：字节/半字载入的对齐与扩展 (MEXT.v)

载入分两级：先按字节偏移把目标字段右移到最低位得到 real_din，再按扩展方式做符号或零扩展。这样存储器只需按 32 位端口返回数据，字段选择与扩展全部在 CPU 内完成，也使同一套逻辑能直接服务于后来的 MMIO 子字读。

```
always @(*) begin // 第一级：按偏移右移
    case (byte_offs)
        2'b01 : real_din = { 8'h0, din[31: 8]};
```

```

        2'b10 : real_din = {16'h0, din[31:16]};
        2'b11 : real_din = {24'h0, din[31:24]};
        default: real_din = din;
    endcase
end
always @(*) begin                                // 第二级: 按 op 扩展
    case (op)
        `RAM_EXT_B : ext = {{24{real_din[ 7]}}, real_din[ 7:0]};
        `RAM_EXT_BU: ext = {24'h0,                real_din[ 7:0]};
        `RAM_EXT_H : ext = {{16{real_din[15]}}, real_din[15:0]};
        `RAM_EXT_HU: ext = {16'h0,                real_din[15:0]};
        default    : ext = real_din;
    endcase
end

```

六、关键实现之四: Booth 乘法器 (multiplier.v)

采用基 2 Booth 算法, IDLE $\rightarrow$ ADD_SUB $\rightarrow$ SHIFT 三态循环, 迭代 WIDTH 次。启动时把乘数放进 product 低位并在最低位补一个 0, 被乘数同时准备正、负两份; ADD_SUB 按 product[1:0] 决定加被乘数、加被乘数的相反数还是不动; SHIFT 做算术右移, 计数到 LAST_COUNT 时按 RESULT_LSB 截取结果位段。每次迭代占 2 拍, 32 位乘法约 64 拍。模块用 WIDTH / RESULT_WIDTH / RESULT_LSB 三个参数化, mul 与 mulh 复用 32 位实例取乘积的低 32 与高 32 位, mulhu 用 WIDTH = 33、RESULT_LSB = 32 的第二个实例实现零扩展高位乘。

```

wire [PRODUCT_WIDTH+1:0] shifted_product =
    {product[PRODUCT_WIDTH+1], product[PRODUCT_WIDTH+1:1]}; // 算术右移
IDLE: if (start) begin
    product      <= {{(WIDTH+1){1'b0}}, y, 1'b0};
    multiplicand <= {x[WIDTH-1], x};
    multiplicand_neg <= ~{x[WIDTH-1], x} + 1'b1;
    state        <= ADD_SUB;
end
ADD_SUB: begin
    case (product[1:0])                // Booth 重编码
        2'b01: product <= {product[PRODUCT_WIDTH+1:WIDTH+1] + multiplicand,
                             product[WIDTH:0]};
        2'b10: product <= {product[PRODUCT_WIDTH+1:WIDTH+1] + multiplicand_neg,
                             product[WIDTH:0]};
        default: product <= product;
    endcase
    state <= SHIFT;
end
SHIFT: begin
    product <= shifted_product;
    if (count == LAST_COUNT) begin
        z <= shifted_product[RESULT_LSB+RESULT_WIDTH:RESULT_LSB+1];
    end
end

```

```

        state <= IDLE;
    end else begin count <= count + 1'b1; state <= ADD_SUB; end
end

```

七、关键实现之五：恢复余数除法器 (divider.v)

除法器采用符号-数值表示的恢复余数 (restoring) 算法, 接口宽度 $WIDTH = 33$, 即 {符号, 32 位数值}。每拍完成一次"余数左移一位并移入商的最高位、与除数比较、够减则减"的迭代, 共 32 拍。商的符号是被除数与除数符号的异或, 余数符号跟随被除数; 当商或余数的数值为 0 时强制输出全 0, 避免出现 -0 这种非规范结果。

```

wire [MAG_WIDTH:0] shifted_remainder = {remainder_mag, quotient_mag[MAG_WIDTH-1]};
wire [MAG_WIDTH:0] extended_divisor = {1'b0, divisor_mag};
wire quotient_bit = shifted_remainder >= extended_divisor;
wire [MAG_WIDTH-1:0] next_remainder = quotient_bit
    ? shifted_remainder[MAG_WIDTH-1:0] - divisor_mag
    : shifted_remainder[MAG_WIDTH-1:0];
wire [MAG_WIDTH-1:0] next_quotient =
    (quotient_mag << 1) | {(MAG_WIDTH-1){1'b0}}, quotient_bit};
// 启动时确定符号; 结束时消除 -0
quotient_sign <= x[WIDTH-1] ^ y[WIDTH-1];
remainder_sign <= x[WIDTH-1];

```

```

z <= quotient_is_zero ? {WIDTH{1'b0}} : {quotient_sign, next_quotient};
r <= remainder_is_zero ? {WIDTH{1'b0}} : {remainder_sign, next_remainder};

```

ALU 侧负责二补码与符号-数值之间的换算, 并单独处理除零: 按 RISC-V 规定, 除零的商为全 1、余数为原被除数。

```

assign div_a_mag = a[31] ? ~a + 32'h1 : a;
assign div_b_mag = b[31] ? ~b + 32'h1 : b;
assign div_quo    = div_quo_sm[32] ? ~div_quo_sm[31:0] + 32'h1 : div_quo_sm[31:0];
`ALU_DIV : c = div_by_zero_r ? 32'hffff_ffff : div_quo;
`ALU_REM : c = div_by_zero_r ? dividend_r    : div_rem;

```

八、关键实现之六：单周期数据通路上的多周期完成握手 (cpu_core.v)

访存与乘除法都不可能在一拍内结束, 因此核内用两个 pending 标志承载在途操作, 并把"本条指令完成"分解成三类互斥事件; 只有 inst_done 有效时 PC 才推进、寄存器才提交。这样数据通路保持单周期形状, 而无界延迟的存储器与多拍的乘除单元都能正确接入。

```

assign normal_done = ifetch_valid & !mem_start & !mul_div_start;
assign mem_done    = mem_pending & (daccess_rvalid | daccess_wresp);
assign mul_div_done = mul_div_pending & !mul_div_busy;
assign inst_done    = normal_done | mem_done | mul_div_done;
assign commit_rf_we = normal_done & rf_we
    | mem_pending & daccess_rvalid
    | mul_div_done;
assign rf_wR = mem_pending | mul_div_pending ? captured_rf_waddr : inst[11:7];

```

由于 ifetch_valid 只有效一拍, 多周期期间指令字已经不可用, 因此在发起访存或乘除的当拍把必要信息锁存下来: 目标寄存器号 captured_rf_waddr、载入扩展方式 captured_load_op、地址低两位 captured_mem_offset。ALU 内部同样用 op_r 锁存当前长延

迟操作码，使结果在指令字撤销之后仍能被正确选出。

指令类别	完成事件	耗时
普通运算 / 分支 / 跳转	normal_done	1 拍
载入 / 存储	mem_done	取决于存储层次, Basic 路径 1 拍, Cache 命中 1 拍, 缺失需整行回填
mul` / `mulh` / `mulhu	mul_div_done	约 64 拍
div` / `divu` / `rem` / `remu	mul_div_done	约 32 拍

1.3 单周期 CPU 仿真分析

要求：在半字/字节访存指令、乘除法指令中分别任选 1 条指令，结合波形截图进行分析。截图需清晰，且需要包含信号名和指令执行所需的关键信号；此外，要结合波形具体时刻具体信号值来分析指令执行过程。

一、波形的获取方式

Trace 框架会为每个用例生成 VCD 波形，落在 cdp-tests/waveform/<用例>.vcd，可用 GTKWave 或 Surfer（仓库内已带 cdp-tests/sim_simple.surf.ron 视图配置）打开；用例的反汇编在 cdp-tests/asm/<用例>.dump，两者对照即可定位到具体 PC。历史 Basic 路径（Inst_ROM + Data_RAM）用配置 single-basic，接入 Cache 与 AXI 后的路径用 single-soc-cache。

```
just trace single-basic sh      # 半字存储用例，生成 cdp-tests/waveform/sh.vcd
```

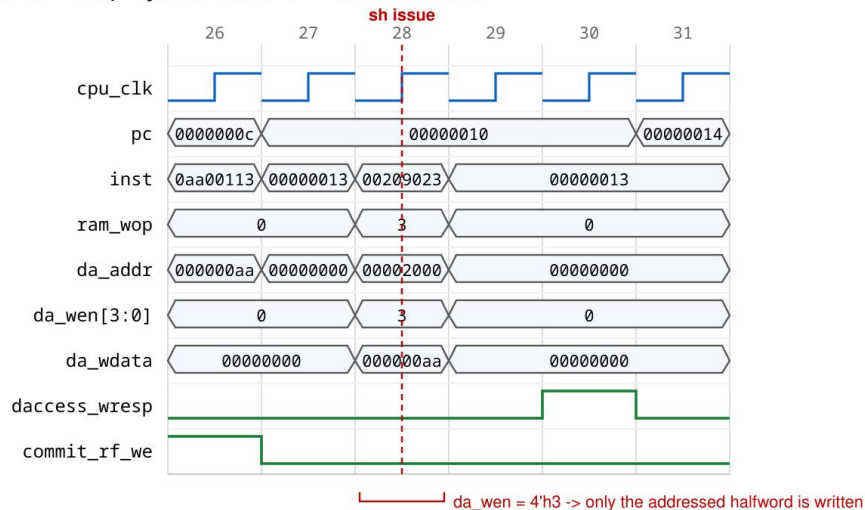
```
just trace single-basic div    # 除法用例，生成 cdp-tests/waveform/div.vcd
```

二、半字访存指令分析：sh（存半字）

选取官方 sh 用例 test_3 段。相关反汇编（cdp-tests/asm/sh.dump）：

```
2c: fffffb137 lui x2, 0xfffffb
30: a0010113 addi x2, x2, -1536      # x2 = 0xfffffaa00
34: 00209123 sh x2, 2(x1)          # x1 = 0x00002000, 写地址 0x00002002
38: 00209703 lh x14, 2(x1)         # 读回并符号扩展，期望 0xfffffaa00
```

sh (store halfword): byte strobe and write handshake



source: cdp-tests/waveform/sh.vcd, config single-basic, posedge-sampled cycle index

图 2 sh 指令的字节选通与写握手波形 (Trace 用例 sh, 配置 single-basic)

按数据通路, PC = 0x34 这一拍应当观察到的信号值如下表。sh 的关键在于地址低两位为 2'b10, MREQ 因此把字节使能左移 2 位、把写数据左移 16 位, 从而只改写 32 位存储字的高半字。

信号	期望值	说明
pc	0x00000034	当前指令地址
inst	0x00209123	sh x2, 2(x1)
ifetch_valid	1	指令返回有效, 仅这一拍为 1
alua_sel` / `alub_sel	ALU_A_RS1` / `ALU_B_EXT	地址 = rs1 + 立即数
ext	0x00000002	S 型立即数 = 2
alu_op` / `alu_c	ALU_ADD` / `0x00002002	有效地址, 低两位为 2'b10
ram_wop	`RAM_WE_H` (4'b0011)	半字写模板
da_wen	4'b1100	4'h3 << 2, 只使能高两个字节
da_wdata	0xaa000000	0xffffaa00 << 16, 把低半字搬到高半字位置
da_ren` / `ram_rop	4'b0000` / `RAM_EXT_N	存储指令不读
mem_start` / `mem_pending	1` / 下一拍为 `1	发起唯一的一次在途访存
rf_we` / `commit_rf_we	0` / `0	sh 不写寄存器
inst_done	0	normal_done 被 mem_start 抑制, PC 保持不变

下一拍存储器接受请求, daccess_wen = 4'b1100、daccess_addr = 0x00002002、daccess_wdata = 0xaa000000。再下一拍 daccess_wresp = 1, 于是 mem_done = 1、inst_done = 1, PC 才推进到 0x38; commit_rf_we 仍为 0, 说明存储指令只改存储器不改寄存器。

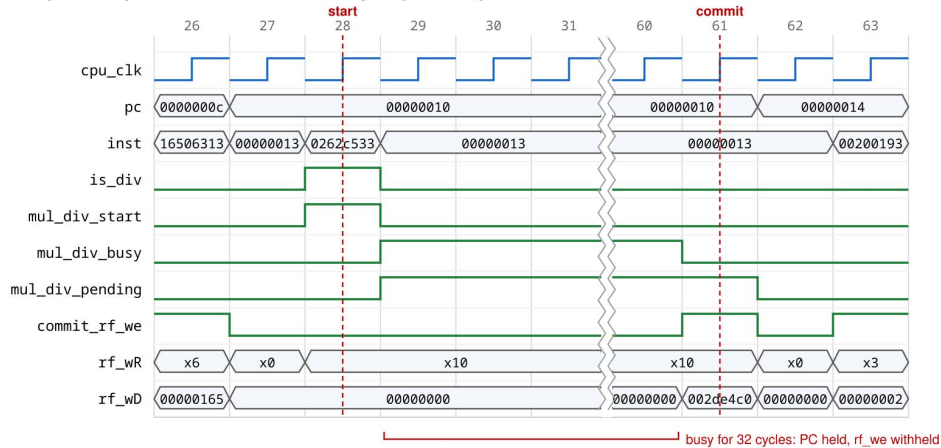
紧随其后的 `lh x14, 2(x1)` 可以在同一张截图里验证读路径: `daccess_rdata` 返回整个 32 位字 (高半字为 `0xaa00`), `MEXT` 先按 `byte_offs = 2'b10` 右移 16 位得到 `real_din = 0x0000aa00`, 再按 `RAM_EXT_H` 以 `real_din[15] = 1` 符号扩展, 得到 `rf_wD = 0xffffaa00`。此时 `mem_pending & daccess_rvalid` 使 `commit_rf_we = 1`, `rf_wR` 取自锁存的 `captured_rf_waddr = 14`。存储与载入一对指令共同证明半字通路的字节选择、移位和符号扩展都正确。

三、乘除法指令分析: `div` (有符号除法)

选取仓库自建的乘除法定向程序 `projects/single_cycle/src/coe/mul_div_test.asm` 的 `test0` 段 (每条指令都带期望值注释, 便于逐点核对):

```
lui t0, 0x40000
ori t0, t0, 0x7B      # t0 = 0x4000007B
ori t1, zero, 0x165   # t1 = 0x00000165
div a0, t0, t1         # 期望 a0 = 0x002DE4C0
rem a0, t0, t1         # 期望 a0 = 0x000000BB
```

div: multi-cycle completion handshake on a single-cycle datapath



source: cdp-tests/waveform/div.vcd, config single-basic; cycles 32..59 elided

图 3 `div` 指令的多周期完成握手波形 (Trace 用例 `div`, 配置 `single-basic`)

`div` 的执行分为三段, 能清楚看到波形启动 1 拍、迭代 32 拍、提交 1 拍。

阶段	时刻	应观察到的现象
启动	<code>'ifetch_valid' = 1</code>	<code>'alu_op' = ALU_DIV (5'h11)</code> , <code>'is_div' = 1</code> , <code>'div_flag' = 1</code> ; ALU 把 <code>a</code> 、 <code>b</code> 转成数值 <code>div_a_mag = 0x4000007B</code> 、 <code>div_b_mag = 0x00000165</code> ; <code>'op_r'</code> 在下一个时钟沿锁存 <code>5'h11</code> , <code>'dividend_r'</code> 锁存 <code>0x4000007B</code> , <code>'div_by_zero_r' = 0</code> (除数非零); <code>'mul_div_pending'</code> 置 1, <code>'inst_done' = 0</code> , PC 保持不变
迭代	启动后 32 拍	<code>'U_div.busy' = 1</code> , <code>'count'</code> 从 0 递增到 31; 每拍 <code>'quotient_mag'</code> 左移一位并把 <code>quotient_bit</code> 移入最低位、 <code>'remainder_mag'</code> 更新为 <code>next_remainder</code> ; <code>'ALU.busy' = 1</code> 使 <code>'inst_done'</code> 保持 0, PC 与 <code>inst</code> 全程不变 (<code>inst</code> 因 <code>ifetch_valid</code> 已撤销而回落为 NOP 编码 <code>0x13</code> , 此时结果由 <code>op_r</code> 而非 <code>op</code> 选出, 这是波形上最值得注意的一点)

提交	<code>`busy` = 0</code>	<code>`div_quo_sm` = {0, 0x002DE4C0}</code> ，符号位为 0 故 <code>`div_quo` = 0x002DE4C0</code> ； <code>`alu_c` = 0x002DE4C0</code> ； <code>`mul_div_done` = 1</code> → <code>`inst_done` = 1</code> 、 <code>`commit_rf_we` = 1</code> ， <code>`rf_wR` = 10 (a0)</code> ， <code>`rf_wD` = 0x002DE4C0</code> ；下一拍 PC 推进
----	-------------------------	--

同一段程序里的 `rem` 用同一台除法器，只是 ALU 改选 `div_rem`；对 `test1` 段的 `-26 / 5` 还应验证符号处理：商 `0xFFFFFFFFB` (`-5`)、余数 `0xFFFFFFFF` (`-1`)，即商符号为两操作数符号异或、余数符号跟随被除数。若把除数改成 0，则 `div_by_zero_r = 1`，波形上 `alu_c` 应直接给出 `0xFFFFFFFF` (`rem` 则给出被除数 `dividend_r`)，且不启动迭代。

四、验证结论

上述两类指令的通路在官方 Trace 全量用例中得到闭环验证：45 个用例（44 条指令用例 + `start` 综合程序）在 `single-basic`、`single-axi-direct-bypass`、`single-axi-direct-cache`、`single-soc-bypass`、`single-soc-cache` 五个单周期配置下均为 45 通过 / 0 失败。Trace 逐条比对提交的 PC、写回寄存器号与写回值，因此半字/字节访存的字段选择与符号扩展、乘除法的数值与符号都被逐指令核对过；乘法另有 `mul_div_test.asm` 的 4 组符号组合 × 7 条指令共 28 个带期望值的测试点作为定向验证。

2 流水线 CPU 及 SoC 设计与实现

2.1 流水线 CPU 数据通路

要求：绘制完整的流水线数据通路图，无需画出模块内的具体逻辑，但要标出模块的接口信号名、模块之间信号线的信号名和位宽，并用文字阐述各模块的功能。数据通路图应当能体现出流水线是如何划分的，并用文字阐述每个流水级具备什么功能、需要完成哪些操作。

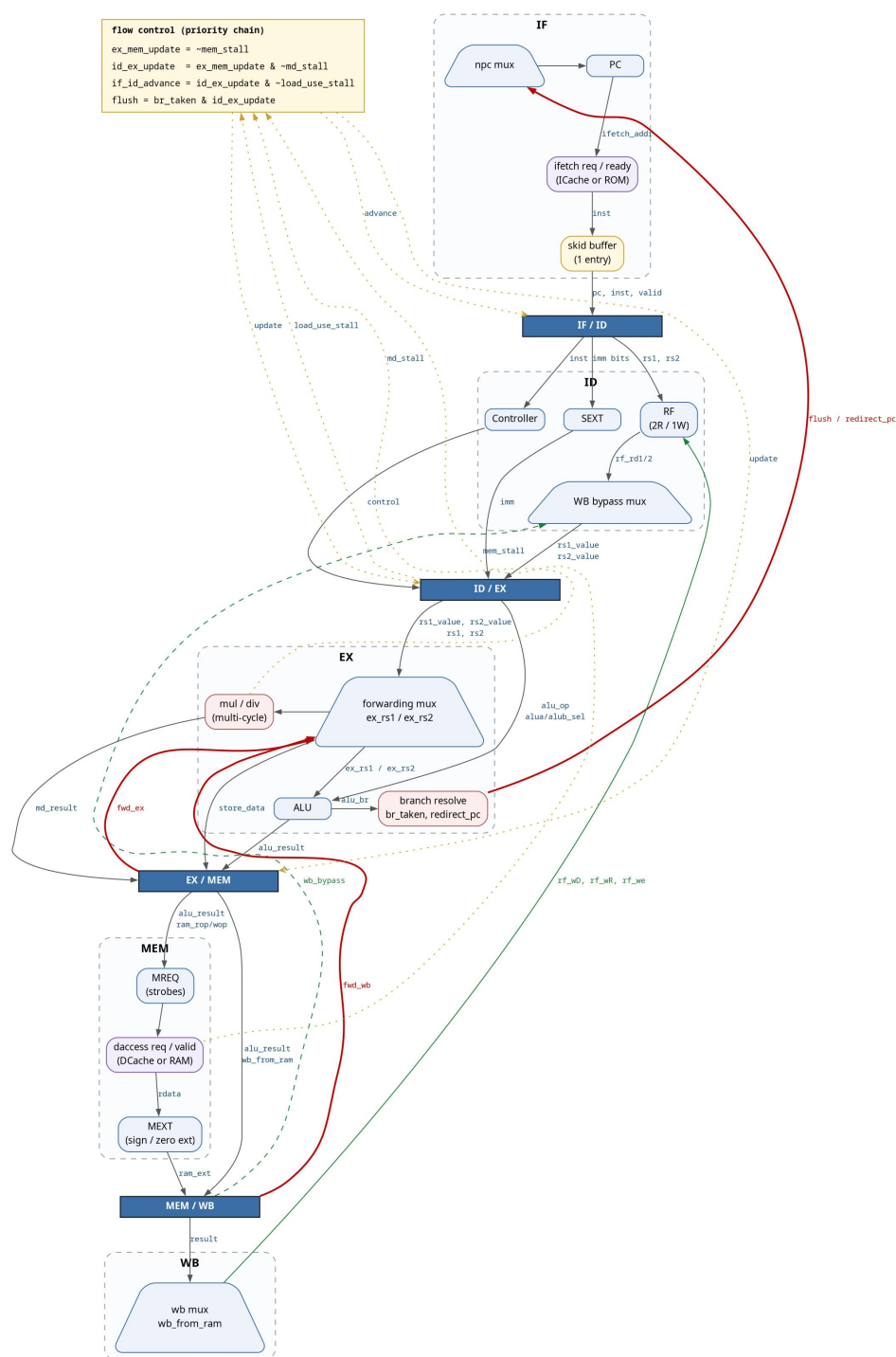


图 4 五级流水线数据通路与流控 / 前递路径

一、流水级划分

设计为经典的五级流水：IF（取指）、ID（译码与取寄存器）、EX（执行与分支解析）、MEM（访存）、WB（写回）。级间由四组段寄存器隔开。与教科书流水线的差别在于：取指与访存都是无界延迟接口，要经过 Cache、AXI 桥和互连，因此 IF 与 MEM 各自只允许一个在途请求，并用暂停信号把整条流水压住；乘除法单元是多拍的，EX 级用暂停等待它。

流水级	完成的操作	级间接口
IF	按 PC 发起取指请求并接收返回的指令；维护 PC 自增与被重定向后的 PC；对被冲刷的在途取指打丢弃标记；后面带一级 skid buffer，使存储器每拍都能答复时取指吞吐达到每拍一条	写 IF_ID (valid / pc / inst)
ID	Controller 组合译码；读寄存器堆两个端口；SEXT 生成立即数；把指令格式中未编码的 rs1/rs2 归一化上报为 x0；WB→ID 读端口旁路；load-use 冒险检测	写 ID_EX (17 个字段)
EX	EX/MEM 与 MEM/WB 两路前递选择操作数；ALU 运算；启动并等待乘除单元；用同一套比较器解析 beq/bne/blt/bge/bltu/bgeu 与 jal/jalr，产生 flush 与 redirect_pc；除 WB_RAM 以外的写回来源在本级就选定	写 EX_MEM (9 个字段)
MEM	MREQ 生成访存请求（字节使能与写数据移位）；等待读/写响应；MEXT 对返回数据做对齐与扩展	写 MEM_WB (5 个字段)
WB	把结果写回寄存器堆（写端口在 ID 级例化，使能为 mem_wb_valid & mem_wb_rf_we），同时作为前递与旁路的最后一级来源	—

二、段寄存器字段表

段寄存器	字段	位宽	说明
IF_ID	valid	1	取指完成后才有效，冲刷时清零
	pc	32	本条指令的 PC
	inst	32	指令字
ID_EX	valid	1	清零即为气泡
	pc	32	随指令携带的 PC
	rs1_value` /`rs2_value	32 / 32	译码读出或旁路得到的源操作数
	imm	32	立即数
	rd	5	目标寄存器号

	rs1` / `rs2	5 / 5	供 EX 前递单元比较的源寄存器号
	npc_op	2	控制转移类型, 由 EX 解析
	alu_op	5	ALU 功能码, 含分支比较
	alua_sel` / `alub_sel	1 / 1	ALU 两个操作数来源
	is_mul` / `is_div	1 / 1	启动乘法 / 除法单元, EX 暂停到 busy 落下
	ram_rop` / `ram_wop	3 / 4	访存类型, 透传到 MEM
	rf_we	1	写回使能, 携带到 WB
	rf_wsel	2	写回来源; 除 WB_RAM 外均在 EX 解析完毕
EX_MEM	valid	1	清零即为气泡
	pc	32	随指令携带的 PC
	alu_result	32	载入/存储的有效地址; 其他指令则已是按 rf_wsel 选好的写回值
	store_data	32	前递之后的待写数据
	rd	5	目标寄存器号
	rf_we	1	写回使能
	wb_from_ram	1	rf_wsel 为 WB_RAM 时置位, 选 MEXT 输出而不是 alu_result
	ram_rop	3	驱动读请求与 MEXT 扩展, MEXT 的字节偏移取 alu_result[1:0]
	ram_wop	4	驱动写字节使能
MEM_WB	valid	1	清零即为气泡
	pc	32	随指令携带的 PC
	result	32	ALU 结果或载入数据
	rd	5	目标寄存器号
	rf_we	1	最终写回使能

三、三个关键设计决定

1. 取指与访存都是无界延迟接口, 因此 IF 与 MEM 各只允许一个在途请求。IF 只在 IF_ID 与其后的 skid buffer 中至多有一个槽位会被占用时才发起新请求; EX 级重定向时, 把已经发出的在途取指打上丢弃标记, 让它返回后被直接丢掉, 而不是错误地写进 IF_ID。
2. 分支、jal 与 jalr 全部在 EX 级用 ALU 已有的比较器解析, 取指侧采用静态"默认不跳转"。因此一次成功的控制转移固定丢弃两条更年轻的指令 (IF 与 ID 各一条), 代价恒为 2 拍; 分支不成立时零代价。
3. 乘除单元按电平启动, EX 在单元启动之后必须撤下操作码, 否则被暂停期间会被反复重启; 结果在 busy 落下的当拍锁存。

另外, 未编码 rs1/rs2 的指令格式 (如 U 型、J 型) 一律把源寄存器号上报为 x0, 避免立即数位被误当成寄存器号污染前递比较器。

2.2 流水线 SoC 详细设计

要求：(1) 结合关键代码详细说明流水线冒险的检测及解决方法。(2) 绘制总线模块的状态转换图，标注清楚状态转移条件及每个状态产生的输出信号；结合关键代码详细说明总线及外设 I/O 接口电路的实现逻辑。

一、流控优先级链

流水线的暂停与冲刷动作不是解耦的而是收敛成一条自上而下的优先级链, 实现上只有三个使能信号 (见 projects/pipeline/src/rtl/cpu_core.v:81-83):

```
wire ex_mem_update = ~mem_stall;
wire id_ex_update = ~mem_stall & ~md_stall;
wire if_id_advance = id_ex_update & ~load_use_stall;
```

这三行的共同语义是越靠后的暂停吞并越靠前的暂停: MEM 一停, EX、ID、IF 全停; EX 因乘除停顿时, MEM 仍可继续排空。下表与 design/pipeline/flow_control.csv 的七行逐行对应, 优先级 0 最高。

条件	优先	IF_ID	ID_EX	EX_MEM	MEM_WB	PC
reset	0	清零	清零	清零	清零	PC_INIT_VAL
`dmem_stall` (MEM 等响应)	1	保持	保持	保持	插气泡	保持
`mul_div_stall` (EX 等乘除)	2	保持	保持	插气泡	推进	保持
flush_on_mispredict	3	清零	清零	推进	推进	重定向
load_use_stall	4	保持	插气泡	推进	推进	保持
`imem_stall` (IF 未取回)	5	插气泡	推进	推进	推进	保持
normal_advance	6	推进	推进	推进	推进	pc+4

二、数据冒险的检测与解决

数据冒险全部靠前递解决, 只有 load-use 这一种必须暂停一拍。

(1) EX 级两路前递及其优先级

EX 级操作数在进入 ALU 前经过两级选择: 优先取 EX/MEM (更年轻的生产者), 其次取 MEM/WB, 都不命中才用段寄存器里锁存的值 (cpu_core.v:292-307):

```
wire fwd_ex_rs1 = ex_mem_valid & ex_mem_rf_we & (ex_mem_rd != 5'h0)
                & (ex_mem_rd == id_ex_rs1);
wire fwd_wb_rs1 = mem_wb_valid & mem_wb_rf_we & (mem_wb_rd != 5'h0)
                & (mem_wb_rd == id_ex_rs1);
wire [31:0] ex_rs1 = fwd_ex_rs1 ? ex_mem_alu_result :
                    fwd_wb_rs1 ? mem_wb_result      : id_ex_rs1_value;
```

三个比较条件缺一不可: `valid` 排除气泡, `rf\_we` 排除不写寄存器的指令 (store、branch), `rd != x0` 排除写 x0 的伪写。EX/MEM 上永远不会挂一个尚未取回的载入值——因为 load-use 检测器已经把这样的消费者拦在 ID 级, 所以这里直接前递 `ex\_mem\_alu\_result` 是安全的。

(2) WB→ID 读端口旁路

寄存器堆是组合读、同步写, WB 的写和 ID_EX 的锁存发生在同一个时钟沿, 因此 ID

级必须看到"正在被写入"的值 (cpu_core.v:236-241):

```
wire wb_bypass_rs1 = mem_wb_valid & mem_wb_rf_we & (mem_wb_rd != 5'h0)
                    & (mem_wb_rd == id_rs1);
```

```
wire [31:0] id_rs1_value = wb_bypass_rs1 ? mem_wb_result : rf_rd1;
```

为了不让立即数位被误当作寄存器号污染上面所有比较器, 未编码 rs1/rs2 的指令格式一律把源寄存器号归一化上报为 x0:

```
wire [4:0] id_rs1 = uses_rs1 ? if_id_inst[19:15] : 5'h0;
```

```
wire [4:0] id_rs2 = uses_rs2 ? if_id_inst[24:20] : 5'h0;
```

(3) load-use 暂停

载入结果要到 MEM/WB 才可用, 所以紧随其后的消费者在 ID 多等一拍, 之后走普通的 MEM/WB→EX 前递 (cpu_core.v:245-249):

```
wire id_ex_is_load = id_ex_ram_rop != `RAM_EXT_N;
```

```
assign load_use_stall = if_id_valid & id_ex_valid & id_ex_is_load
```

```
                    & (id_ex_rd != 5'h0)
```

```
                    & ((id_ex_rd == id_rs1) | (id_ex_rd == id_rs2));
```

(4) 被暂停的 EX 必须每拍重捕获操作数

这是本设计里最反直觉的一处。EX 被暂停时, 它的生产者仍在继续下移, 因此"当前可见的前递来源"会在等待期间消失。解决办法是被 hold 的 ID_EX 每拍把当拍算出的 'ex_rs1/ex_rs2' 写回自己 (cpu_core.v:276-280 的 else 分支):

```
end else begin
```

```
    id_ex_rs1_value <= ex_rs1;
```

```
    id_ex_rs2_value <= ex_rs2;
```

```
end
```

(5) 乘除单元的电平启动与撤码

乘除单元按电平启动, 若 EX 被暂停而操作码不撤下, 单元会被反复重启。因此用 'md_launched'/'md_captured' 两个标志把"启动—等待—锁存"三段分开, 并在启动后把送往 ALU 的操作码换成 'ALU_ADD' (cpu_core.v:309-316, 341-344):

```
wire ex_is_md = id_ex_valid & (id_ex_is_mul | id_ex_is_div);
```

```
wire md_present = ex_is_md & ~md_launched & ~md_captured;
```

```
wire md_capture = md_launched & ~mul_div_busy;
```

```
assign md_stall = ex_is_md & ~md_captured;
```

```
wire ex_holds_md_op = id_ex_is_mul | id_ex_is_div;
```

```
wire [4:0] alu_op_in = (ex_holds_md_op & ~md_present) ? `ALU_ADD
                    : id_ex_alu_op;
```

注意撤码判据用的是 'id_ex_is_mul|id_ex_is_div' 字段本身而不是 'ex_is_md': 气泡仍然携带上一条指令的译码结果, 若按 'ex_is_md' 门控, 一个无效级就能在活跃运算背后重启单元并拿走它的结果。

三、控制冒险的检测与解决

分支、jal、jalr 全部在 EX 级解析, 取指侧采用静态"默认不跳转" (cpu_core.v:373-378):

```
wire br_taken = id_ex_valid
```

```
                    & ((id_ex_npc_op == `NPC_JMP)
```

```
                    | (id_ex_npc_op == `NPC_JALR)
```

```

| ((id_ex_npc_op == `NPC_BRA) & alu_br));
assign flush      = br_taken & id_ex_update;
assign redirect_pc = npc;

```

‘flush’ 与 ‘id_ex_update’ 相与, 保证只在这条转移指令真正离开 EX 的那一拍才冲刷, 否则被 MEM 暂停期间会反复重定向。一次成功转移固定丢弃 IF 与 ID 各一条指令, 代价恒为 2 拍; 分支不成立时零代价。另外, 被冲刷时可能已经有一次在途取指发出, 它带上 ‘fetch_discard’ 标记, 返回后被直接丢弃而不写入 IF_ID。

四、总线状态转换

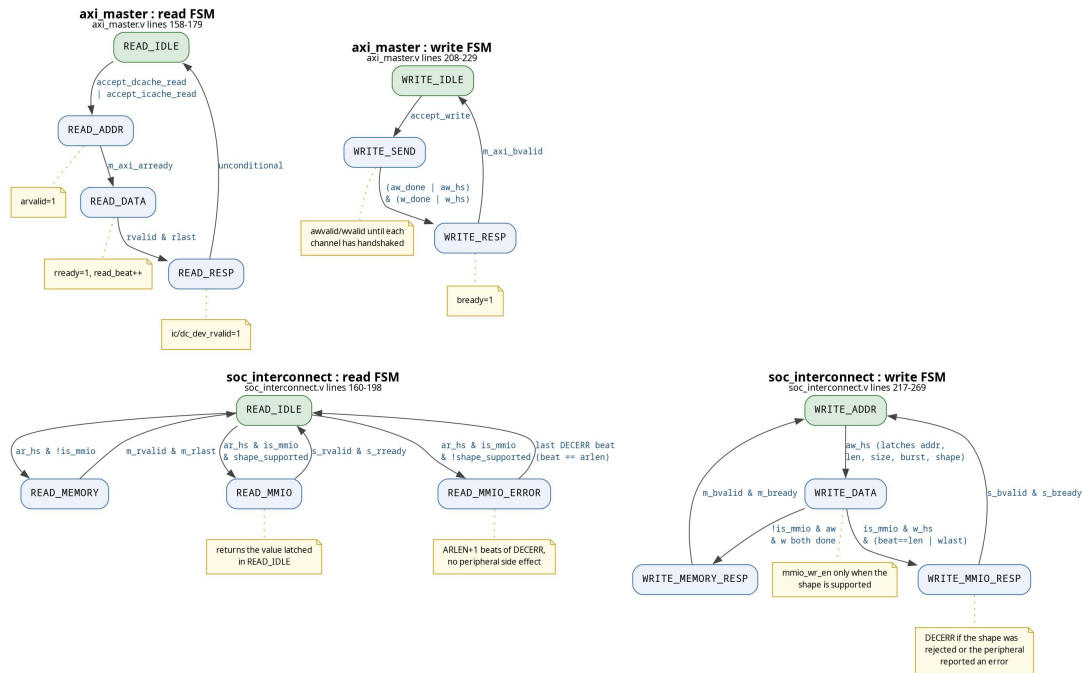


图 5 axi_master 与 soc_interconnect 的四台读 / 写状态机

(1) axi_master: Cache 侧请求到 AXI4 主接口

桥内有两台独立 FSM, 读通道四态、写通道三态。仲裁优先级为 DCache 写 > DCache 读 > ICache 读, 且只在两台 FSM 同时空闲 (‘bridge_idle’) 时才接受新请求, 因此桥上永远只有一笔在途事务。

状态	转移条件	该状态的输出
READ_IDLE	accept_dcache_read` 或 `accept_icache_read` → `READ_ADDR	锁存 read_addr / read_len / read_source , read_beat 清零
READ_ADDR	m_axi_arready` → `READ_DATA	arvalid = 1, araddr/arlen/arsize/arbust 保持
READ_DATA	m_axi_rvalid && m_axi_rlast` → `READ_RESP	rready = 1, 按 read_beat 把每拍 rdata 拼入 128 位行缓冲, read_beat 自增
READ_RESP	无条件 → ‘READ_IDLE’	向发起方 (ICache 或 DCache) 回 rvalid 与 整行数据
WRITE_IDLE	accept_write` → `WRITE_SEND	锁存 write_addr / write_data / write_strobe

WRITE_SEND	地址与数据两个握手都完成 → `WRITE_RESP`	awvalid = ~write_addr_done , wvalid = ~write_data_done, 两条通道各自握手后置 done
WRITE_RESP	m_axi_bvalid` → `WRITE_IDLE	bready = 1, 向 DCache 回写响应

burst 长度的决策是本模块的关键点：普通 Cache 缺失回填要取整条 128 位行，而 uncached 的 MMIO 访问只能取一个 32 位端口值，绝不能发四拍突发。

```
read_len <= dc_cpu_raddr[31:16] == 16'hffff
    ? 8'h0 : `DC_BLK_LEN - 1;
```

(2) soc_interconnect: 地址空间判决与 MMIO 单拍通路

互连接 `addr[31:16] == 16'hffff` 把事务分流到 MMIO 或主存 BRAM。读、写各一台四态 FSM。

状态	转移条件	该状态的输出
READ_IDLE	地址握手后：非 MMIO → `READ_MEMORY`；MMIO 且形状合法 → `READ_MMIO`；MMIO 但形 状非法 → `READ_MMIO_ERROR`	仅在 "MMIO 且形状合法" 时才拉高 mmio_rd_en, 非法形状不产生任何外设副 作用
READ_MEMORY	主存返回最后一拍 → `READ_IDLE`	把主存读通道透传给发起方，支持四拍突 发
READ_MMIO	s_axi_rvalid && s_axi_rready` → `READ_IDLE`	rdata = 外设读值, rresp = mmio_read_resp, rlast = 1 (单拍)
READ_MMIO_ERROR	错误拍计满 → `mmio_error_read_len` → `READ_IDLE`	rdata = 0, rresp = 2'b11 (DECERR), 按请 求的 arlen 补齐同样拍数, rlast 只在最后 一拍为 1
WRITE_ADDR	地址握手 → `WRITE_DATA`	锁存 write_addr / write_len / write_is_mmio 与形状合法性 write_shape_supported
WRITE_DATA	MMIO → `WRITE_MMIO_RESP`; 主存 → `WRITE_MEMORY_RESP`	仅在 write_data_is_supported 时才拉高外 设写使能
WRITE_MEMORY_RESP	s_axi_bvalid && s_axi_bready` → `WRITE_ADDR`	透传主存写响应
WRITE_MMIO_RESP	同上	bresp = OKAY 或 DECERR

合法形状的判据是 AXI 侧的三个字段加子字选通的组合：

```
wire read_shape_supported = s_axi_arlen == 8'h0 &&
    s_axi_arsize == 3'd2 &&
```

```
s_axi_arburst == 2'b01;
```

写还要额外检查选通与地址低两位是否自洽，这就是 MMIO 支持真实 sb/sh 的契约（`valid\_mmio\_strobe` 函数）：偏移 0 允许 4'b0001 / 4'b0011 / 4'b1111，偏移 1 只允许 4'b0010，偏移 2 允许 4'b0100 / 4'b1100，偏移 3 只允许 4'b1000。其余 4×16 组合一律判为非法。

两条设计纪律贯穿这台互连：其一，**形状检查通过之前不产生任何本地使能**，被拒的事务必须零副作用（一次被拒的 LED 写不能真把 LED 改掉，一次被拒的 UART RX 读不能真把 FIFO 弹出）；其二，**错误响应的形状必须与请求一致**，即按请求的 arlen 补齐同样拍数、只在最后一拍拉 rlast，否则发起方会挂死在等 rlast 上。

五、外设 I/O 接口电路

地址译码分三级：互连接 `addr[31:16]` 判 MMIO；MMIO 汇总层按 `addr[31:12]` 选设备；设备内按 `addr[11:2]` 选寄存器。所有外设读均为单拍组合返回，写均为单拍接受。

外设	方向	接口电路
拨码开关	只读	板上引脚经两级触发器同步（打 `ASYNC_REG` 属性）后寄存，消除跨时钟域亚稳态；读回为纯组合选择
LED	只写	写数据按 AXI 的 wstrb 逐字节合并进保持寄存器（byte-strobe merge），未被选通的字节保持原值，从而 sb 可以只改一个字节
数码管	只写	同样的 byte-strobe 合并写入显示值寄存器；seven_segment 模块用一个 19 位计数器，取其高 3 位作扫描相位，轮流点亮 8 位数码管并输出对应的段码与位选
UART	读写	四个寄存器（状态、控制、发送数据、接收数据）+ 收发各一个 16 深 FIFO；TX FSM 按波特率分频依次移出起始位、8 位数据、停止位；RX FSM 在起始位下降沿对齐后在每位中点采样。读接收寄存器会弹出 FIFO，因此它是一个有副作用的读，必须受形状检查保护
计时器	只读	64 位自增计数器，映射为高低两个 32 位寄存器；软件按“读高位—读低位—再读高位”的协议判断中途是否发生低位翻转，翻转则重读，从而在 32 位总线上得到一致的 64 位快照

2.3 流水线 SoC 仿真分析

一、波形的获取方式

流水线产品的仿真同样通过 Trace 框架与系统仿真套件产生 VCD 波形，配置由 config/build-configs.tsv 选定：`pipeline-basic` 用历史 ROM/RAM 路径看纯流水线行为，`pipeline-soc-cache` 用完整 SoC 路径看 Cache 与 AXI 行为。

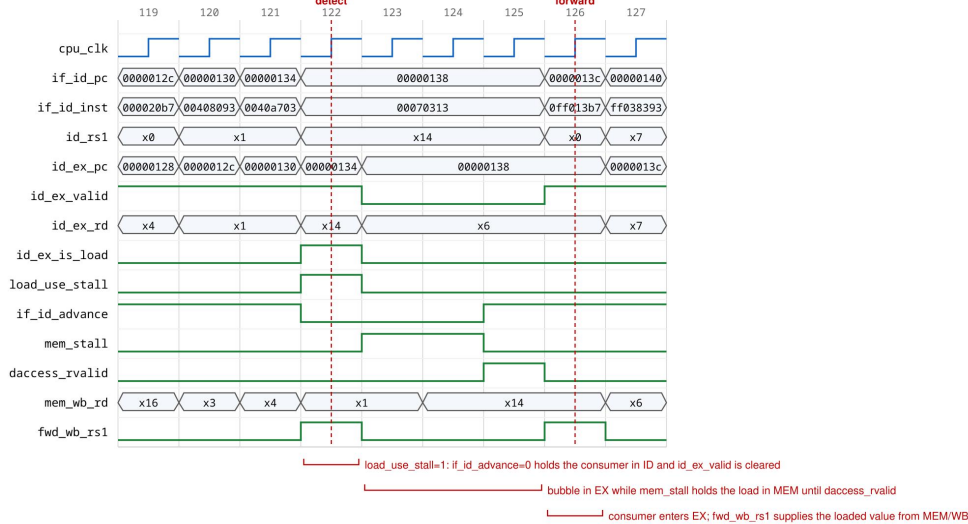
```
just trace pipeline-basic addi          # 观察冒险与前递，波形在 cdp-tests/waveform/
just trace-all pipeline-soc-cache      # 45 个 AXI Trace 用例
just system coremark                   # pipeline-soc-cache 上的 CoreMark 系统仿真
```


二、数据冒险波形分析

考察一段"背靠背相关 + load-use"的典型序列。设指令流为：

```
lw    x5, 0(x10)    # A: 载入
addi  x6, x5, 1      # B: 与 A 相关, 构成 load-use
add   x7, x6, x5      # C: 与 B 相关, 走 EX/MEM->EX 前递
```

load-use hazard: hold IF/ID, inject a bubble, then forward from MEM/WB (lw @0x134 -> addi @0x138)



source: cdp-tests/waveform/lw.vcd, config pipeline-basic. EX/MEM never holds a pending load value, so one bubble is enough and no second stall is needed.

图 6 load-use 数据冒险：暂停 IF/ID、插入气泡、再由 MEM/WB 前递（配置 pipeline-basic）

波形上应当逐拍看到下面的现象。表中 T 为 B 进入 ID 的那一拍。

时刻	应观察到的现象
T	A 在 EX, B 在 ID。`id_ex_is_load` = 1、`id_ex_rd` = 5、`id_rs1` = 5, 因此 `load_use_stall` = 1。IF_ID 保持不变 (B 不前进), ID_EX 下一拍插入气泡 (`id_ex_valid` = 0)
T+1	A 进入 MEM, ID_EX 是气泡, B 仍留在 ID。`load_use_stall` 落为 0 (`id_ex_is_load` 此时属于气泡, 且 `id_ex_valid` = 0)
T+2	A 进入 MEM/WB, B 进入 EX。`fwd_wb_rs1` = 1 (`mem_wb_rd` = 5 == `id_ex_rs1`), `ex_rs1` 取 `mem_wb_result` 即载入值, 而不是段寄存器里那个陈旧的 `id_ex_rs1_value`。这一拍证明 load-use 只花了 1 拍代价
T+3	B 进入 MEM 并已在 EX_MEM 上给出结果, C 进入 EX。`fwd_ex_rs1` = 1 (`ex_mem_rd` = 6), `ex_rs1` 取 `ex_mem_alu_result`; 同时 `fwd_wb_rs2` = 1 (`mem_wb_rd` = 5), `ex_rs2` 取 `mem_wb_result`。两路前递同拍生效, 且两条通路各自独立选择, 是前递优先级正确的直接证据

若把 B 换成 `mul x6, x5, x5`, 则同一张波形还能看到暂停的相互作用: `md\_stall` 拉高约 32 拍, 期间 `ex\_mem\_update` 仍为 1 (MEM 继续排空), 而 `id\_ex\_update` 与 `if\_id\_advance` 均为 0; 并且被 hold 的 ID_EX 每拍重捕获 `ex\_rs1/ex\_rs2`, 因此即使 A 在等待期间从 MEM/WB 退出, B 拿到的操作数依然正确。

三、Cache 缺失到总线回填的波形分析

取指缺失是 SoC 路径上最完整的一条握手链: CPU → ICache → axi_master → 互连 → BRAM → 回填 → 返回。以一次 ICache 读缺失为例, 波形应当依次出现下列事件。

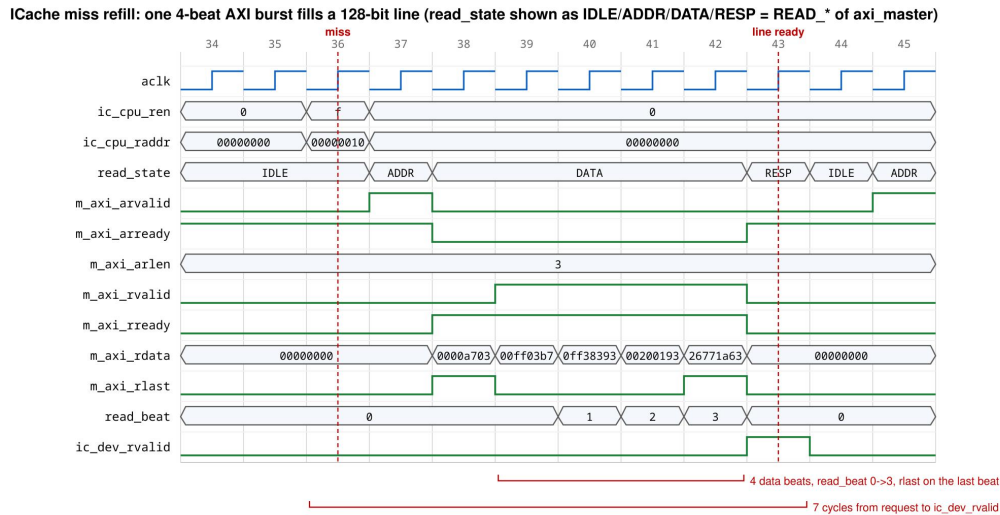


图 7 ICache 缺失回填：一次 4 拍 AXI 突发填满一条 128 位缓存行（配置 pipeline-soc-cache）

时刻	应观察到的现象
T	'ifetch_req' = 1, ICache 比较 tag 后判定缺失, 'ifetch_valid' 保持 0, 'ifetch_ready' 撤下 (不再接受新请求)
T+1	ICache 向桥发起整行读: 'ic_cpu_ren' = 4'hf, 'ic_cpu_raddr' 为行对齐地址; 桥此时 'bridge_idle' = 1 且无 DCache 请求, 于是接受, 'ic_dev_rdy' 给出接受指示, 读 FSM 进入 READ_ADDR
T+2	'm_axi_arvalid' = 1, 'm_axi_arlen' = 8'd3 (四拍突发, 整条 128 位行), 'arsize' = 3'd2, 'arburst' = 2'b01 INCR。互连判 'araddr[31:16] != 16'hffff', 进入 READ_MEMORY 并把请求透传给主存
T+3 起连续 4 拍	'm_axi_rvalid' = 1, 'rready' = 1, 'read_beat' 从 0 计到 3, 四拍数据依次拼入 128 位行缓冲; 第四拍 'm_axi_rlast' = 1, 'rresp' 全程为 2'b00 OKAY。读 FSM 转入 READ_RESP
末拍	桥把整行一次交回: 'ic_dev_rvalid' = 1, 'ic_dev_rdata[127:0]' 为完整行。ICache 同拍写入数据阵列与 tag/valid, 并**在同一拍**把请求字选出, 'ifetch_valid' = 1 送回 CPU——回填与返回不分成两拍, 这是取指优化的一部分
末拍+1	'ifetch_ready' 恢复为 1。若下一条指令命中同一行, 则可立即背靠背答复, 配合 IF_ID 后的 skid buffer 达到每拍一条取指

对照实验值得一并截图：把地址换成 MMIO 区 ('addr[31:16] == 16'hffff'), DCache 走 uncached 通路, 此时 'arlen' 必须为 8'h0 单拍、互连进入 READ_MMIO, 'rlast' 与 'rvalid' 同拍拉高。这条对照正是修复"MMIO 读被当成四拍突发因而被互连判 DECERR"缺陷之后必须保持的性质。

四、实测性能数据与优化

以下数据来自 'pipeline-soc-cache' 配置上单次 CoreMark 迭代的系统仿真。停顿归因计数器互斥, 即每一个被浪费的周期只被归入一类。

提交	改动	cycles / IPC	主要停顿变化
e8dc5a4	只加测量: cycles /	1,692,827 /	md = 650,422; ifetch = 457,689; mem =

	retired / 停顿归因计数器	0.192	258,034
f405c53	Booth 加法与移位融合为一拍, 32 位乘法约 64 拍降到约 32 拍	1,382,771 / 0.236	md 650,422 → 340,374 (总周期 -18.3%)
ad6319c	取指每拍一次请求: 'ifetch_ready' 握手、ICache 命中/回填可背靠背答复、IF_ID 后一级 skid buffer	1,112,667 / 0.293	ifetch 457,674 → 138,554 (总周期 -19.5%)

累计周期数下降 34.3%, IPC 从 0.192 提升到 0.293。优化后剩余停顿预算为 md = 335,292 (约 30%)、mem = 258,013 (约 23%)、ifetch = 138,554、load-use = 22,330。两条剩余的杠杆(基-4 Booth、把访存地址路径改为组合)都是用组合延迟换周期数,而设计另有 50 MHz 的时钟下限要求,因此在取得 Vivado 时序数据之前没有继续优化——只靠周期数无法判断这种交换是否净收益。

五、验证结论

流水线产品在五个配置 ('pipeline-basic'、'pipeline-axi-direct-bypass'、'pipeline-axi-direct-cache'、'pipeline-soc-bypass'、'pipeline-soc-cache') 下均通过全部 45 个官方 Trace 用例 (45 通过 / 0 失败), 即流水线 CPU 不只通过 Basic Trace, 在 AXI 直连与完整 SoC (含 Cache、MMIO 互连、五类外设) 两种拓扑、Cache 旁路与启用两种形态下都全绿。CoreMark 的系统仿真校验三个与迭代次数无关的 CRC, 全部通过。需要如实声明的边界: 上述全部为仿真证据。流水线产品的 Vivado 物理工程尚未补齐 SoC fabric 文件集, 因此流水线 SoC 的综合、实现、时序收敛与板级运行均未进行, 本报告不给出这部分数据。

3 设计过程中遇到的问题及解决方法

问题一：MMIO 读被当成四拍突发，被互连判为 DECERR 而表面功能正常

现象：Cache 启用后，读拨码开关这类外设寄存器在程序上看结果是对的，但完整链路波形里出现了 `RRESP=DECERR`。

定位：`axi\_master` 对每个 DCache 读都固定设置 `ARLEN=3`（按 `DC\_BLK\_LEN=4` 回填整行），没有区分 `0xFFFF\_xxxx` 的 uncached 地址；而 `soc\_interconnect` 把非零 `ARLEN` 的 MMIO 事务判为 DECERR。链路上真实发生的是 `ARLEN=3 → RRESP=DECERR → DCache 仍然收下了开关数据`。之所以“功能正常”，纯粹因为 `axi\_master` 当时**完全忽略 `RRESP`**，把错误响应里的数据照样交给了 Cache。

未发现原因：三层测试各自都有盲区——外设单元测试直接驱动 MMIO 端口，从不经过 AXI；互连测试手写 `ARLEN=0`；Cache 与 AXI master 的独立测试里根本没有外设请求。三种测试都不假，但没有一种把“Cache + 桥 + 互连 + 外设”接成完整链路。

修复：先补一个**会失败**的完整链路测试把缺陷钉住，然后让 `axi\_master` 对 uncached 地址发单拍读：

```
read_len <= dc_cpu_raddr[31:16] == 16'hffff
    ? 8'h0 : `DC_BLK_LEN - 1;
```

回归：新增的链路测试从失败转为通过，并断言有效外设访问必须收到 `RRESP=OKAY`；全部 Trace 与单元/集成套件保持全绿。

教训：这是本项目最有价值的一次调试。“功能看起来对”和“协议正确”是两件事；一个被忽略的响应字段可以把协议错误伪装成正确行为。此后所有分层测试都被要求显式写出“这一层不能证明什么”。

问题二：被拒绝的 MMIO 事务已经产生了副作用

现象：跨模块审查时构造非法事务，发现互连在返回 DECERR 的同时真的改动了外设状态。两个具体复现：一次非法的双拍 LED 写返回 DECERR，同时把 LED 值改成了 `0xCAFE`；一次非法形状的 UART 接收寄存器读会真的把接收 FIFO 弹出一个字节。此外，对非零 `ARLEN` 的读只返回一拍带 `RLAST` 的响应，响应形状与请求不一致。

定位：互连的读/写通路在完成 `ARLEN`、`ARSIZE`、`ARBURST` 与选通合法性检查之前就拉高了本地外设读写使能。错误响应形状不一致则是因为错误路径没有按请求的拍数补齐。

修复：把本地使能改成只在形状检查通过时才产生——

```
assign mmio_rd_en = read_state == READ_IDLE && read_address_handshake &&
    read_address_is_mmio && read_shape_supported;
```

并新增 `READ\_MMIO\_ERROR` 状态，按请求的 `ARLEN+1` 拍逐拍返回 DECERR，只在最后一拍拉 `RLAST`，保证响应形状与请求一致，发起方不会挂死在等 `RLAST` 上。

回归：补了“被拒绝事务不得产生 LED / UART / FIFO 副作用”的失败用例；`soc\_stage3\_tb` 对四种地址低两位 × 全部 16 种选通共 64 种组合做了穷举，确认合法组合正常、其余组合稳定返回 DECERR 且零副作用。

问题三：被暂停的 EX 级使用了会消失的前递值

现象：取指提速之后，CoreMark 出现零星的结果错误，而在提速之前同样的 RTL 一

直是对的。

定位：ID_EX 的操作数是在 ID 阶段捕获的。当 EX 被暂停（例如等乘除或等访存）时，它的生产者仍在继续下移；如果生产者在 EX 获准离开之前就退出了 EX/MEM 与 MEM/WB，那个能满足它的前递就消失了，EX 只能用段寄存器里那个尚未更新的旧值。这个缺陷长期被“每两拍才取一条指令”的低取指吞吐掩盖——流水线太松，暂停期间生产者几乎不会跑掉。

修复：被 hold 的 ID_EX 每拍把当拍算出的前递结果写回自己：

```
end else begin
    id_ex_rs1_value <= ex_rs1;
    id_ex_rs2_value <= ex_rs2;
end
```

回归：Trace 全量恢复通过，CoreMark 的 CRC 校验通过。

教训：性能优化会改变时序耦合，从而暴露原本被掩盖的功能缺陷。所以性能提交必须和功能回归绑在一起跑，不能只看周期数。

问题四：取指吞吐被卡在两拍一条

现象：加上停顿归因计数器后测得，CoreMark 的 1,692,827 个周期里有 457,689 拍（约 27%）花在等取指上，明显偏高。

定位：原设计只有在上一次取指已经返回**并且** IF_ID 已经排空之后才允许发起下一次请求，于是即使存储器每拍都能答复，取指吞吐也被硬性限制在每两拍一条；ICache 命中时也不能背靠背答复。

修复：三处配套改动——(1) 增加 `ifetch\_ready` 握手，让 ICache 显式表达“本拍可以接受新请求”；(2) ICache 命中与回填都在同一拍把请求字返回，回填与返回不再分成两拍；(3) 在 IF_ID 之后加一级单项 skid buffer，使取指答复即使在 IF_ID 暂时不能推进时也有落点。发起条件因此变成“两个指令槽位至多一个会被占用时才发起”，并对被冲刷路径上的在途取指打丢弃标记。

回归：取指停顿 457,674 → 138,554 拍，总周期 -19.5%；连同上一次的乘法器优化，累计周期数下降 34.3%，IPC 从 0.192 提升到 0.293。五个流水线配置的 45 个 Trace 用例全部保持通过。

4 总结

要求：(1) 学完本课程后的个人收获；(2) 对本课程的建议和意见。请在**认真总结和思考后**填写。

一、个人收获

最大的收获不是"会写 CPU 了", 而是学会了**先定合同, 再写 RTL**。本项目把设计文档做成了硬门禁: ``design/single_cycle/{datapath,control_signals}.csv`` 与 ``design/pipeline/{stage_registers,hazards,flow_control}.csv`` 必须在对应 RTL 之前填完, 空值表示"尚未设计", ``-`` 表示"该字段不施加语义约束", 而有副作用的控制信号 (``npc_op``、``rf_we``、``ram_rop``、``ram_wop``、``is_mul``、``is_div``) 永远不允许填 ``-``。这套约定看起来只是文档规范, 实际效果是把"我以为它应该是这样"变成了一张可以逐行比对的表。流水线段寄存器 35 个字段、流控七行优先级表都是先在 CSV 里定好、RTL 照着写, 因此在实现阶段几乎没有出现过"两个模块对同一个信号理解不一致"的返工。

第二个收获是**对验证边界的敏感**。问题一 (MMIO 被当作四拍突发) 给我的冲击最大: 外设单元测试、互连测试、Cache/AXI 测试三层全绿, 缺陷却真实存在, 因为没有任何一层把完整链路接起来, 而 ``axi_master`` 忽略 ``RRESP`` 又把协议错误伪装成了正确功能。此后我在每层测试旁边都显式写下"这一层不能证明什么", 并且把跨层的完整链路测试当作独立的一层。相应地, 报告里也严格区分仿真证据与物理证据——流水线 SoC 的综合、实现、时序与板级运行确实没有做, 就明确写没有做, 而不是用单周期 SoC 的数据顶替。

第三个收获是**性能优化必须先有测量**。第一版流水线跑 CoreMark 用了 1,692,827 拍、IPC 只有 0.192, 但当时完全说不清时间花在哪里。加上互斥的停顿归因计数器之后, 答案一目了然: 乘除 650k、取指 458k、访存 258k。据此做了两次针对性优化 (Booth 加法与移位融合、取指每拍一次请求), 累计降低 34.3% 周期、IPC 提升到 0.293。同样重要的是知道**何时停手**: 剩下的两条杠杆 (基-4 Booth、组合访存地址路径) 都是用组合延迟换周期数, 而设计另有 50 MHz 时钟下限, 只看周期数无法判断这种交换是不是净收益, 因此在拿到时序数据之前没有继续改。顺带还学到一课: 那次取指提速暴露了一个被低取指吞吐长期掩盖的前递缺陷 (问题三), 说明性能提交必须和功能回归绑定执行。

第四点是工程化的价值。用一张配置矩阵 (``config/build-configs.tsv``) 驱动全部十个稳定配置, 两个产品各自只有一份 RTL 真相源, 拓扑与 Cache 的选择全部通过编译宏完成而不是复制文件; 所有验证入口收敛到一个 ``just`` 命令表。这套东西前期投入不小, 但它让"改一处、全量回归"变成一条命令的事, 也是最后能有信心声称"五个流水线配置 45/45 全绿"的原因。

二、课程建议

建议一: 把"分层验证的边界"提前讲。课程给了很好的 Trace 框架, 但 Trace 是端到端的黑盒比对, 它能告诉你错了, 不太能告诉你错在哪一层。如果在 AXI 与 Cache 章节之前先讲清"单元测试 / 集成测试 / 端到端测试各自能证明什么、不能证明什么", 并给一个"三层全绿但缺陷存在"的反例, 学生会少走很多弯路——我自己就是靠踩坑才补上这一课的。

建议二: 给性能要求配一套推荐的测量口径。要求里有 CoreMark 优化目标, 但没有规定测量方法。周期数、IPC、CoreMark/MHz 三个指标的分母各不相同, 很容易报出互相矛盾的数字 (我在整理数据时就遇到过同一次运行按不同分母算出两个"分数"的情况)。如果课程直接给出建议的计数器口径 (至少包括总周期、退休指令数、按类互斥的停顿归因) 和统一的报告格式, 不同同学之间的优化效果才具备可比性。

建议三：明确区分"仿真通过"与"下板通过"的评分口径。物理下板依赖具体板卡与烧录环境，而仿真闭环是完全可自动化的。如果评分表能把这两类证据分列，学生就更容易诚实地标注哪些结论有物理证据、哪些只有仿真证据，而不必为了凑齐一张表去含混其词。

5 AI 大模型使用日志

一、AI 的使用方式与参与边界

本项目从一开始就把 AI 定位为**设计讨论者与实现草稿的提供者**，而不是结论的来源。分工是固定的：ISA 语义、数据通路与控制信号取值、暂停与前递的优先级，先由我们自己在设计 CSV 里定稿；AI 主要用在三件事上——在已定合同下写或改一段 RTL/脚本草稿、对症状明确的缺陷提出定位假设、以及做代码审查并反问“这一层测试证明不了什么”。

采纳判据只有一条：**能否归约为一条可执行命令，并且这条命令实际变绿**。凡是落不到 `just lint <config>`、`just trace <config> <case>`、`just unit/integration/system <suite>` 上的建议，都不作为结论写进 RTL 或报告。对于缺陷类问题，顺序进一步被固定为“先补一个会失败的测试把缺陷钉住，再改实现，最后看该测试由红转绿”，而不是以模型的自信程度作为依据。

第二条边界是**证据归属**。仿真闭环可以完全自动化，物理下板不行。因此 Vivado 综合实现、bitstream 烧写与 EGO1 板上现象观察全部由我们本人执行并记录，AI 不允许声称任何它没有实际运行过的验证；报告中“仿真通过”与“下板通过”因此严格分列。第三条边界是**最小改动**：每一行改动都必须能追溯回当前任务本身，不顺手“优化”相邻代码、不重构没坏的东西，只清理由本次改动产生的孤立信号与参数。

二、Harness 工程化：把仓库改造成可判定的协作环境

我们把“与 AI 协作”当成一个工程问题：模型只是回路中的一个环节，回路的可靠性取决于它周围的上下文、工具与判据。为此在仓库里搭了一套脚手架（harness），由五部分组成。

(1) **常驻上下文合同**。根目录的 AGENTS.md (CLAUDE.md 是指向它的符号链接) 是每次进入仓库首先被读取的文件，它写明：项目范围为完整的 EGO1 + miniRV 作业，不得自行缩小到最低及格路径；必须先读的文件顺序 (README.md → docs/workflow.md → 当前任务书 → 指导书对应章节 → 相关源码与测试)；工作风格（先讲假设、简单优先、外科手术式改动）；RTL 硬约束（保持 miniRV_SoC.U_cpu.U_core 层次与命名、复位 PC 为 0x0000_0000、不得擅自改动 RUN_TRACE 与 `/* verilator public */` 相关代码、不得在 cdp-tests/mySoC/ 下写代码、不得直接编辑 Windows Vivado staging 目录）；以及一张“改动级别 → 必需验证”的门禁表。合同越明确，模型自由发挥的空间越小，结果越可重复。

(2) **单一公开入口**。所有验证收敛到根 Justfile (它只是 scripts/build.sh 的门面)：doctor、status、show-config、lint、trace、trace-all、unit、integration、system、program、gate、vivado。仓库刻意不保留根 Makefile，closure 门禁会在它重新出现、或 README/workflow 里重新出现 make 命令时直接失败。这样 AI 不需要猜测“怎么编译、怎么跑用例”，也无法发明一条只在它自己语境里成立的命令。

(3) **显式配置矩阵**。config/build-configs.tsv 固定十个稳定配置（两个产品 × basic / AXI 直连 / 产品 SoC × bypass / cache），每行锁定 product、topology、memory_model、cache、backend、编译宏与产物目录；<config> 永远显式给出，从不默认。拓扑与 Cache 的切换只经由编译宏，不允许改 defines.vh 或复制文件。这条规则直接消灭了“为了让某个用例通过而悄悄 fork 一份 RTL”这类捷径。

(4) **前置设计门禁与外部黄金判据**。design/single_cycle/{datapath,control_signals}.csv 与 design/pipeline/{stage_registers,hazards,flow_control}.csv 必须在对应 RTL 之前填完（空值表示尚未设计，“-”表示该字段不施加语义约束，而 npc_op、rf_we、ram_rop、ram_wop、is_mul、

is_div 这些有副作用的控制信号永远不能是"-"); cdp-tests/ 提供的 45 个官方 Trace 用例是外部黄金判据, 失败时用生成的 VCD 对照 cdp-tests/asm/<case>.dump 定位。判据来自课程框架与仓库, 而不是来自模型的自我评价。

(5) **目标驱动的执行循环与聚合门禁**。每个任务先转写成"步骤 → 验证"对, 然后让循环自己跑到全绿, 而不是每改一次就停下来确认。分层套件之上再叠聚合门禁, 最严的一条是:

just gate closure

它一次跑完 just --fmt --check、doctor、全部单元套件 (cache、axi-master、pipeline-control、peripherals、c-test-software、stage5-contract)、集成套件 (fabric-mmio、dcache-mmio)、十个配置各自的 lint 与全量 Trace、SoC smoke、C_TEST 0-2 与流水线 CoreMark 系统仿真、两个 .xpr 的 XML 校验, 以及 git diff --check。环境本身也被固定: 日常门禁在 Linux devcontainer 中运行, 指导书与 Trace 框架以 pinned submodule 引入并显式升级。正因如此, "改一处、全量回归"才真的只是一条命令, 也才敢在报告里声称五个流水线配置 45/45 全绿。

配套的还有过程留痕: docs/devlog/ 下每份任务书写明范围、顺序与验收口径, artifacts/ 下保留的报告记录来源 commit 与工具版本, materials/MANIFEST.md 记录受限课程材料的 SHA-256 provenance。这些文件既是给人看的, 也是给模型看的上下文。

三、有效性与失败模式

最有价值的用法不是"帮我写一个模块", 而是"在已有症状上给定位假设"和"找出测试的盲区"。第 3 章问题一正是这样解决的: 外设单元测试、互连测试、Cache/AXI 测试三层全绿, 缺陷却真实存在; AI 给出的定位是 axi_master 对所有 DCache 读固定发 ARLEN=3、未区分 0xFFFF_xxxx 的 uncached 地址, 而互连把非零 ARLEN 的 MMIO 事务判为 DECERR, "功能看起来正常"只是因为桥忽略了 RRESP。这个假设随后被一个刻意写成会失败的完整链路测试钉死, 修复后由红转绿——是这个顺序而不是模型的措辞构成了采纳理由。两次性能优化 (Booth 加法与移位融合、取指每拍一次请求) 同样遵循"先有互斥的停顿归因计数器给出测量, 再让 AI 提方案, 最后用周期数与全量 Trace 验收"。

失败模式集中在两类。一是上下文不完整时给出局部自治但违反全局合同的方案, 例如绕过配置矩阵直接改宏定义、或触碰 Trace 契约相关代码——这类由 AGENTS.md 的硬约束与 closure 门禁直接拦下。二是把"仿真能过"当成"设计正确", 尤其是在被忽略也不会立刻报错的协议字段上 (RRESP、RLAST 就是典型)。综合两类失败, 我们的结论是: **模型输出的可靠性上限, 取决于它周围判据的强度**; 把判据工程化——可执行、可重复、显式覆盖分层边界——比反复追问模型有效得多。

四、提示词与采纳情况要点摘录

下表为整理的要点摘录, 按实际工作节点归纳。使用的模型为 Claude (Anthropic) 与 GPT-5.6-Sol (OpenAI)。所有 AI 给出的方案都经过本人核对, 并以 `just lint` / `just trace` / 单元与集成测试的实际结果作为采纳依据。

序号	提示词 (要点)	回答要点与采纳情况
1	单周期 Booth 乘法器现在是 IDLE / ADD_SUB / SHIFT 三态, 32 位乘法要 64 拍。能否把加法与移位合并到一拍	建议把加/减的结果直接接到移位逻辑上, 一拍内完成"根据 product[1:0] 决定加减 → 右移一位", 状态机退化为 IDLE / RUNNING 两态。采纳, 乘法降到约 32 拍, CoreMark 总周期降 18.3%
2	取指停顿占了 27%, 原因是上一次取指返回且 IF_ID 排空后才能发下	给出三处配套改动: ICache 增加 ifetch_ready 握手、命中与回填在同一拍返回请求字、IF_ID 之后加一级

	一次请求。怎么做到每拍一条	单项 skid buffer; 并提醒被冲刷路径上的在途取指必须打丢弃标记。采纳, 取指停顿 457,674 → 138,554 拍
3	完整链路波形里 MMIO 读出现 RRESP = DECERR, 但程序结果是对的, 帮我定位	定位为 axi_master 对所有 DCache 读固定发 ARLEN=3, 未区分 0xFFFF_xxxx uncached 地址, 而互连把非零 ARLEN 的 MMIO 判为 DECERR; "功能正常"是因为桥忽略了 RRESP。同时指出应当先补一个会失败的链路测试再改代码。采纳
4	流水线的前递、load-use 暂停与乘除暂停应该怎么组织, 优先级怎么定	建议把暂停收敛成一条"越靠后的暂停吞并越靠前"的优先级链 (只需三个使能信号), 并提示两个易错点: 未编码的 rs1/rs2 要归一化为 x0 以免污染比较器; 乘除按电平启动, 撤码判据必须用段寄存器字段而不是 valid。采纳, 对应 flow_control.csv 的七行表
5	帮我梳理 Verilator lint 与 Trace 的工作流, 怎么让"改一处、全量回归"成为一条命令	建议用一张配置表 (product / topology / cache / backend / defines) 驱动全部构建, 拓扑与 Cache 用编译宏而非复制文件切换, 验证入口统一收敛到 Justfile, 并对 Trace 的共享 obj_dir 加文件锁串行化。采纳, 即当前的 config/build-configs.tsv 与 just 命令表